

AGENCYOS

PHASE 4 - PROTOTYPE AGENT

Task 2: Full UI Design + UI Prototype

RESPONSIBILITIES + BUILD CONTRACT + VERSIONING + QA HANDOFF + IMPLEMENTATION + TESTING SPECIFICATION

Official Role	UI Prototype Agent / Prototype Build Agent
Phase	Phase 4 - Task 2
Primary Mission	Convert the exact client-approved UI version into a reviewable interactive prototype without silently becoming the Phase 5 production-development system
Primary Input	Locked approved UI version + screen/flow baseline + design tokens/assets + prototype scope
Primary Output	Versioned review prototype/build + source-version traceability + known limitations + QA-ready evidence
Review Order	Prototype Agent → Prototype QA → Admin → PM → Client
Finance Boundary	Prototype Agent does not trigger/verify M2 directly; Task 2 completion workflow triggers Finance after final client approval
Core Rule	Build success ≠ prototype completion; QA + Admin + client approval are required

This document defines the complete Phase 4 Prototype Agent role, including exact input contracts, build types, interaction scope, mock-data policy, version/build metadata, security boundaries, QA handoff, Admin/client revision loops, failure recovery and production-readiness testing.

1. Prototype Agent Mission

The Prototype Agent creates a controlled, versioned, reviewable prototype from the exact approved UI baseline. Its job is to demonstrate the intended screens, navigation, interactions and critical user flows so Admin and Client can validate the product experience before full production development starts.

The Prototype Agent is not the Phase 5 Development Agent. It must not silently build the complete backend, production business logic, production integrations or unapproved features simply because a prototype can technically include them.

2. Start Conditions

- ☐ Phase 4 Full UI Design is complete enough for prototype creation.
- ☐ Design QA has passed the exact UI version.
- ☐ Admin has approved the exact UI version.
- ☐ Client has explicitly approved the exact UI version.
- ☐ Approved UI version is locked and uniquely identifiable.
- ☐ Prototype scope/critical flows are defined.
- ☐ Required design tokens/assets are available.
- ☐ Any required platform/build credentials are available or represented as explicit blockers.

Prototype creation must not begin from a draft, QA-failed, Admin-unapproved, client-unapproved or ambiguous UI baseline.

3. Required Prototype Inputs

- ☐ project_id / organization_id / active phase/task.
- ☐ locked ui_version_id.
- ☐ source Figma/design file/page/node/version references.
- ☐ screen inventory and required states.
- ☐ approved user flows / navigation rules.
- ☐ design tokens, components, icons and assets.
- ☐ platform target(s): Web / Android / iOS preview / desktop / other agreed target.
- ☐ prototype build mode and review scope.
- ☐ mock/test data policy.
- ☐ required validation/error/loading/success states.
- ☐ known exclusions and prototype limitations.
- ☐ approved project branding.
- ☐ revision policy and current prototype revision count.

4. Prototype Agent Responsibility Boundaries

Prototype Agent owns	Prototype Agent must NOT own
Prototype project/build initialization	Final production backend architecture
Screen routing and navigation	Production database design unless prototype specifically needs temporary mock structure
Interactive UI behavior	Real payment settlement unless explicitly approved as sandbox/demo flow
Critical review flows	Production deployment
Mock/sample data and safe local/test state	Production secrets
Input validation needed to demonstrate UX	Full Phase 5 business-rule implementation
Loading/empty/error/success states	Admin/client approval authority
Build/version metadata	QA self-certification
Artifact generation: URL/APK/preview as applicable	M2 payment verification
Fixes after OA/Admin/client-approved revisions	Silent scope expansion

If a requested prototype behavior requires real production logic not approved for Phase 4, the Prototype Agent must raise a structured blocker/clarification rather than inventing Phase 5 behavior.

5. Supported Prototype Output Types

Project type	Possible prototype output	Notes
Web / SaaS / Website	Hosted preview, staging URL or local review build	Use review-safe test environment; no production secrets
Android app	APK / emulator-ready debug or review build	Signing/distribution may be manual/external depending on setup
iOS app	Simulator build / configured preview/distribution path	Respect Apple account/signing limitations
Desktop app	Local packaged review build or preview	Only if the project requires it
Cross-platform	Platform-specific builds generated from common approved UI source	Each build still carries exact source_ui_version

The concrete output type is project-specific. The system should not hardcode Android/APK as the only prototype format.

6. What the Prototype Must Demonstrate

- ☐ Critical screen-to-screen navigation.
- ☐ Primary user journeys.
- ☐ Representative form/input behavior.
- ☐ CTA behavior.
- ☐ Back/close/cancel behavior where important.
- ☐ Required validation states.
- ☐ Loading/empty/error/success states where important to review.
- ☐ Role-specific screen differences where approved.
- ☐ Responsive/platform behavior included in prototype scope.
- ☐ Meaningful content hierarchy using representative/mock data.
- ☐ Approved visual theme/tokens/components.
- ☐ No accidental placeholder screens that contradict approved UI.

7. Mock / Sample Data Policy

- ☐ Prototype may use representative mock/sample data when real backend integration is not required.
- ☐ Mock data must be clearly separated from production data.
- ☐ No production credentials, customer secrets or live payment information inside the prototype.
- ☐ Mock data should be realistic enough to exercise layout and user flow.
- ☐ Edge cases such as empty list, long text, large numbers and missing image should be represented when relevant.
- ☐ If the client needs a live integration demonstration, it must be explicitly approved and configured as sandbox/test wherever possible

Rule: a prototype may simulate backend behavior, but it must not mislead the client into believing a simulated integration is production-ready.

8. Exact UI Source Traceability

- ☐ Every PrototypeBuild stores source_ui_version_id.
- ☐ Build metadata stores source design/Figma references.
- ☐ Build creation rejects non-approved/non-locked UI version.
- ☐ If UI is revised, a new prototype build/revision must be created.
- ☐ A previous client-shared build remains historical.
- ☐ QA/Admin/client decisions reference exact build ID/version.
- ☐ Prototype Agent cannot silently pull 'latest UI' if latest differs from the approved locked source.

9. Prototype Agent End-to-End Flow

CLIENT-APPROVED UI LOCKED → PROTOTYPE BUILD REQUESTED → INPUT VALIDATION → BUILD PLAN → PROJECT/ENV PREP → SCREEN/ROUTE IMPLEMENTATION → INTERACTIONS/STATES → MOCK/TEST DATA → BUILD → SELF-CHECK → PROTOTYPE BUILD READY → QA → DEFECTS? → FIX → QA RETEST → QA PASS → ADMIN REVIEW → EDIT? → FIX → QA → ADMIN → APPROVED → PM SHARE → CLIENT FEEDBACK → ALLOWED REVISION? → NEW BUILD → QA → ADMIN → PM RE-SHARE → CLIENT FINAL APPROVAL → BUILD LOCKED → TASK 2 COMPLETION ELIGIBLE

10. Detailed Prototype Agent Responsibilities

10.1 Input Validation

- ☐ Verify exact locked UI version exists.
- ☐ Verify required screens/flows are available.
- ☐ Verify project/platform target.
- ☐ Verify required assets.
- ☐ Identify external/manual blockers before building.
- ☐ Do not infer missing business logic.

10.2 Build Planning

- ☐ Translate approved screen/flow baseline into prototype routes/screens/components.
- ☐ Identify reusable components.
- ☐ Define mock/test data sources.
- ☐ Define critical interactions and states.
- ☐ Record prototype limitations before review.

10.3 Project / Environment Preparation

- ☐ Reuse existing project/repository structure where appropriate.
- ☐ Avoid creating duplicate prototype stacks when one valid stack exists.
- ☐ Isolate prototype/test configuration from production.
- ☐ Keep environment variables/secrets out of client artifacts.

10.4 UI Implementation

- ☐ Match approved UI structure, tokens, typography, spacing and component intent.
- ☐ Implement all prototype-required screens.
- ☐ Implement navigation and route guards needed for the review flow.
- ☐ Implement states and representative content.

10.5 Interaction Implementation

- ☐ Implement important buttons, links, tabs, drawers, forms, dialogs and navigation.
- ☐ Avoid decorative controls that do nothing if the client is expected to review them.
- ☐ For intentionally non-functional items, label or document the limitation.

10.6 Prototype Build Creation

- ☐ Create versioned build/artifact.
- ☐ Store build ID/version, source UI version, platform/type, environment, artifact ref and generation timestamp.
- ☐ Run safe local/self-checks before QA handoff.

10.7 QA Handoff

- ☐ Submit exact build to QA with source UI version and prototype coverage list.
- ☐ Provide known limitations and test instructions.
- ☐ Do not mark QA PASS.

10.8 QA Defect Fixing

- ☐ Receive structured defect tied to exact build.
- ☐ Fix only the relevant defect/approved impact area.
- ☐ Create revised build when needed.
- ☐ Run focused self-check.
- ☐ Return to QA for retest.

10.9 Admin Edit Handling

- ☐ Receive Admin edit request through workflow.
- ☐ Apply approved change against correct build/UI baseline.

- ☐ Create new build/version.
- ☐ Return to QA before Admin re-review.

10.10 Client Revision Handling

- ☐ Receive PM-created revision request tied to exact client-shared build.
- ☐ Implement only allowed correction/revision.
- ☐ Do not implement possible scope change until governed approval exists.
- ☐ Create new build → QA → Admin before client re-share.

10.11 Final Build Lock

- ☐ After explicit client final approval, mark exact prototype build as final/locked through controlled workflow.
- ☐ Preserve older builds and evidence.
- ☐ Prepare clean Phase 5 handoff references.

11. Prototype Build State Machine

PLANNED → INPUT_VALIDATION → BUILDING → BUILD_READY → QA_REVIEW → QA_CHANGES_REQUIRED / QA_PASS → ADMIN_REVIEW → ADMIN_EDIT / ADMIN_APPROVED → CLIENT_REVIEW → CLIENT_CHANGE / CLIENT_APPROVED → LOCKED

State	Meaning
PLANNED	Build requested from an approved UI baseline
INPUT_VALIDATION	Source UI, assets, platform, flows and blockers checked
BUILDING	Prototype implementation/build in progress
BUILD_READY	Artifact exists and passed internal self-check
QA_REVIEW	Independent Prototype QA active
QA_CHANGES_REQUIRED	Defects require Prototype Agent revision
QA_PASS	Exact build passed QA
ADMIN_REVIEW	Human Admin reviews QA-passed build
ADMIN_EDIT	Admin-approved edit cycle required
ADMIN_APPROVED	Build eligible for PM/client share
CLIENT_REVIEW	Client reviewing exact build
CLIENT_CHANGE	Allowed revision or scope classification required
CLIENT_APPROVED	Client explicitly approved exact build
LOCKED	Final build preserved as Phase 4 baseline

12. Prototype Agent Data Contracts

Entity	Minimum fields
PrototypeBuild	id/version, project_id, organization_id, source_ui_version_id, build_type/platform, artifact_ref, environment, status, limitation_summary, created_at
PrototypeRouteCoverage	build_id, screen_id, route/interaction, covered yes/no, notes
PrototypeTestData	build_id, dataset/source type, production_data=false, reset behavior
PrototypeRevision	from_build_id, to_build_id, origin QA/ADMIN/CLIENT, request/evidence, change summary, status
PrototypeQAReport	build_id, checks, defects, evidence, result, retest refs
ArtifactRecord	build_id, kind URL/APK/package/screenshot/log, storage ref, hash/metadata, access policy
PrototypeBlocker	build_id, blocker type, owner, reason, evidence expected, resume condition
AuditEvent	actor/agent, action, build/entity, before/after, project/tenant, timestamp

13. Admin Panel - Prototype Visibility

Admin surface	Must show
Prototype Overview	Current build, source UI version, status, platform/type, owner, blocker, next action
Build History	All builds/revisions and from→to lineage
Coverage	Which screens/routes/interactions are implemented in each build
Artifacts	Review URL/APK/package and generated evidence
Known Limitations	Explicit prototype limitations/exclusions
Prototype QA	Defects, fix/retest history and exact QA PASS build
Admin Review	CONFIRM/EDIT decision with comments
Client Review	Exact build shared, feedback, final approval
Revision Timeline	QA/Admin/client-origin revisions
Blockers	Missing credentials/provider/build/distribution dependencies
Activity/Audit	Build creation, revisions, QA approvals and locks

Admin must be able to identify exactly which UI version produced which prototype build and what the client actually reviewed.

14. Prototype QA Handoff Package

- ☐ Exact build_id/version.
- ☐ Exact source_ui_version_id.
- ☐ Artifact URL/APK/package.
- ☐ Platform/type and target environment.
- ☐ Prototype coverage list.
- ☐ Critical flows to test.
- ☐ Known limitations.
- ☐ Mock/test credentials/data instructions if safe.
- ☐ Expected loading/error/success states.
- ☐ Build logs/metadata where useful.
- ☐ Any manual setup required to launch the prototype.

15. Security & Permission Boundaries

- ☐ Prototype Agent cannot approve QA/Admin/client gates.
- ☐ Prototype Agent cannot verify payment or open Phase 5.
- ☐ Prototype build cannot access another project/tenant's data.
- ☐ Prototype artifacts must not contain production secrets.
- ☐ Prototype review links should use project-appropriate access controls.
- ☐ Uploads/assets must be validated and scoped.
- ☐ Debug logs exposed to client must not leak tokens, credentials or sensitive data.
- ☐ If authentication is simulated, label it as prototype behavior unless real auth is explicitly part of the prototype scope.
- ☐ Privileged build/revision/lock actions are audited.

16. Cost, Reuse & Build Efficiency

- ☐ Reuse approved Phase 4 UI components/tokens/assets.
- ☐ Reuse the existing prototype project/stack when valid.
- ☐ Small revision should not rebuild unrelated modules unnecessarily where tooling supports incremental builds.
- ☐ Retry should not create duplicate expensive generation/build jobs.
- ☐ Use deterministic build tooling instead of AI for work that code/tooling can perform.
- ☐ Use AI reasoning for ambiguous implementation mapping only when needed.
- ☐ Track build/tool/model usage where AgencyOS telemetry supports it.
- ☐ Preserve cached dependencies/build outputs where safe.

17. Failure & Recovery Handling

Failure	Required behavior
Locked UI version missing	Block build; show exact missing dependency
Asset missing	Block affected screen/flow or use approved placeholder only if allowed
Build command fails	Persist failure/log; no fake BUILD_READY
Dependency install fails	Retry safely / surface actionable blocker
Figma/design source unavailable	Use stored approved source metadata or explicit blocker; no guessing
Artifact upload fails	Build may remain local-ready but not client-share eligible until artifact is accessible
QA worker fails	Preserve build; QA retries independently
Admin requests edit	Create new build/version; return to QA
Client requests revision	Use PM-created revision request; create new build
Client asks scope change	Do not implement until governed approval
Provider/account credential missing	Create external/manual blocker and continue independent work
Duplicate build event	Return/reuse existing build identity where safe

18. Prototype Agent Technical Implementation Plan

PA4-I01 Existing Prototype Stack Discovery

Inspect current Prototype Agent, repositories, build tooling, platform targets, artifact storage, QA, Admin review and Phase 5 handoff before creating new infrastructure.

PA4-I02 Prototype Agent Contract

Define input/output schemas, supported build types, allowed actions, state mutations, permissions and blocker codes.

PA4-I03 Approved UI Input Guard

Implement server-side validation that exact UI source is client-approved and locked.

PA4-I04 Prototype Build Domain

Implement/extend PrototypeBuild, revisions, artifact records, blockers, route coverage and audit metadata.

PA4-I05 Build Planning / Coverage

Translate approved screens/flows into prototype route/interaction coverage.

PA4-I06 Project/Environment Preparation

Reuse project stack; isolate prototype/test config and mock data from production.

PA4-I07 Screen & Navigation Implementation

Implement required screens/routes/navigation from exact approved UI.

PA4-I08 Interaction & State Implementation

Implement critical forms, CTAs, dialogs and required UI states.

PA4-I09 Artifact Generation

Generate versioned preview/APK/package and persist artifact metadata.

PA4-I10 QA Handoff

Create exact-build QA package and structured defect intake.

PA4-I11 QA Revision Loop

Defect→fix→new/revised build→QA retest.

PA4-I12 Admin Edit Loop

Admin EDIT→Prototype Agent→QA→Admin.

PA4-I13 Client Revision Loop

PM-created client revision→Prototype Agent→QA→Admin→PM re-share.

PA4-I14 Final Build Lock

Lock exact final client-approved prototype and preserve history.

PA4-I15 Admin Visibility

Expose build lineage, coverage, artifacts, QA, decisions, limitations and blockers.

PA4-I16 Security / Access

Harden project/tenant isolation, artifact access, secrets and audit.

PA4-I17 Reliability / Idempotency

Prevent duplicate build/artifact/revision jobs and support safe retry.

PA4-I18 Testing / Regression

Run prototype-specific tests and regression against Phase 4 UI/PM/Finance shared workflow.

19. Prototype Agent Test Matrix

Area	Required coverage
Input guard	approved/locked UI, stale UI, missing UI, wrong project UI
Build plan	screen coverage, route coverage, critical-flow coverage
Build types	configured web/APK/iOS/other target behavior as applicable
Mock data	no production data/secrets, realistic edge cases, reset behavior
Navigation	critical routes, back/cancel, deep links where applicable
Interactions	buttons, forms, validation, dialogs, drawers, tabs
States	loading, empty, error, success, permission/offline where applicable
Artifact	version, source UI link, storage/access, checksum/metadata where supported
QA loop	defect, fix, revised build, QA retest
Admin loop	Confirm/Edit and mandatory re-QA after edit
Client loop	exact build share, allowed revision, scope-change rejection
Revision history	from/to build, origin, evidence, old builds preserved
Security	cross-project/tenant denial, no secrets, controlled review access
Recovery	build failure, dependency failure, artifact upload failure, duplicate event
Idempotency	retry does not create unintended duplicate build/revision
Regression	approved UI consistency and shared Task 2 workflow remain intact

20. Mandatory Prototype Agent Test Cases

ID	Scenario	Expected result
PA4-T001	Build request with approved locked UI	Accepted; exact source_ui_version stored
PA4-T002	Build request with draft UI	Rejected
PA4-T003	Build request with QA-failed UI	Rejected
PA4-T004	Build request with Admin-approved but client-unapproved UI	Rejected
PA4-T005	Build request using UI from another project	Denied
PA4-T006	Missing required design asset	Block or approved fallback path
PA4-T007	Critical screen missing from build plan	Coverage fails
PA4-T008	Required route missing	Prototype not QA-ready
PA4-T009	Button expected to navigate but does nothing	Prototype QA defect
PA4-T010	Form validation required	Validation behavior visible
PA4-T011	Loading/empty/error state required	State available for review
PA4-T012	Mock data used	No production record/secret present
PA4-T013	Web prototype build succeeds	Versioned artifact stored
PA4-T014	Android APK target succeeds where configured	Versioned APK artifact stored
PA4-T015	Build process fails	Status FAILED/BLOCKED; no BUILD_READY
PA4-T016	Artifact upload fails	Not client-share eligible
PA4-T017	QA defect tied to build	Revision/fix targets exact build
PA4-T018	Prototype Agent marks own QA PASS	Denied
PA4-T019	QA retest passes revised build	Exact new/revised build becomes QA PASS
PA4-T020	Admin EDIT	New build/revision → QA required
PA4-T021	Try client share before Admin approval	Blocked
PA4-T022	Client requests allowed correction	PM revision request creates new build cycle

ID	Scenario	Expected result
PA4-T023	Client requests new feature	Prototype Agent does not implement; scope-change route
PA4-T024	Revision limit exceeded	No uncontrolled auto-build; escalation
PA4-T025	Client explicitly approves final build	Final build eligible for lock
PA4-T026	Attempt mutate locked final build	Rejected/new revision required
PA4-T027	Duplicate PrototypeBuildRequested event	No unintended duplicate build
PA4-T028	Worker crashes during build	Persisted state supports safe retry
PA4-T029	Cross-tenant artifact access	Denied
PA4-T030	Build contains provider/API secret	Security FAIL
PA4-T031	Old build remains after new revision	Historical lineage preserved
PA4-T032	Phase 5 handoff reads final build	Gets exact locked final prototype

21. Mandatory Prototype Agent E2E Scenario

- ☐ Start with a client-approved locked UI version.
- ☐ Prototype build request is created with exact UI reference.
- ☐ Prototype Agent validates inputs and generates build plan.
- ☐ Implement all critical prototype screens/routes/interactions.
- ☐ Use approved mock/test data.
- ☐ Create versioned build artifact.
- ☐ Submit exact build to Prototype QA.
- ☐ QA finds one broken route.
- ☐ Prototype Agent fixes it and creates revised build.
- ☐ QA retests and passes revised build.
- ☐ Admin reviews and requests one allowed edit.
- ☐ Prototype Agent creates next revision.
- ☐ QA retests; Admin approves.
- ☐ PM shares exact build to client.
- ☐ Client requests one allowed correction.
- ☐ PM creates revision request.
- ☐ Prototype Agent applies correction and creates new build.
- ☐ QA passes; Admin approves; PM re-shares.
- ☐ Client explicitly approves final exact build.
- ☐ Build is locked; old builds remain historical.
- ☐ Phase 4 completion workflow can reference exact final build.

22. Manual / External Dependencies

- ☐ Android/iOS signing credentials may require secure manual/external setup.
- ☐ Apple/Google distribution account steps may remain human-controlled.
- ☐ External review URL hosting/domain may require configured infrastructure.
- ☐ Real sandbox third-party integrations may require provider credentials/account approval.
- ☐ If required build tooling is unavailable in execution environment, record exact blocker and continue all independent work.
- ☐ Human Admin/client approvals remain outside Prototype Agent authority.

23. Prototype Agent Definition of Done

- ☐ Prototype Agent accepts only exact approved/locked UI baseline.
- ☐ Build plan covers all required prototype screens/critical flows.
- ☐ Required navigation/interactions/states are implemented.
- ☐ Mock/test data is separated from production.
- ☐ No production secrets are embedded.
- ☐ Versioned build/artifact metadata is persisted.
- ☐ Source UI version is traceable from every build.
- ☐ Prototype QA handoff contains complete evidence/instructions.
- ☐ QA defect/fix/retest loop works.
- ☐ Admin EDIT cycle forces new QA before re-review.
- ☐ Client revision cycle uses PM-created request and exact build lineage.
- ☐ Possible scope changes are not silently implemented.
- ☐ Final client-approved build can be locked.
- ☐ Historical builds/revisions remain visible.
- ☐ Admin Panel exposes build/QA/review/blocker history.
- ☐ Idempotency, recovery, security and project/tenant isolation tests pass.
- ☐ Phase 5 handoff resolves the exact final prototype build.
- ☐ No unresolved Prototype Agent Phase 4 P0/P1 defects remain.

24. Master Prototype Agent Checklist

A. Inputs

- ☐ locked UI version
- ☐ scope/flows
- ☐ screen inventory
- ☐ tokens/assets
- ☐ platform target
- ☐ prototype mode
- ☐ revision policy

B. Planning

- ☐ route map
- ☐ screen coverage
- ☐ interaction coverage
- ☐ mock/test data
- ☐ limitations
- ☐ dependencies

C. Build

- ☐ screens
- ☐ navigation
- ☐ forms/actions
- ☐ states
- ☐ responsive/platform behavior
- ☐ artifact/version

D. Traceability

- ☐ source UI version
- ☐ Figma/design refs
- ☐ build ID
- ☐ artifact
- ☐ from/to revisions
- ☐ audit

E. QA

- ☐ QA package
- ☐ defects
- ☐ fixes
- ☐ revised build
- ☐ retest
- ☐ PASS

F. Admin Review

- ☐ exact build
- ☐ limitations
- ☐ QA evidence
- ☐ Confirm/Edit
- ☐ re-QA after edit

G. Client Revision

- ☐ PM revision request
- ☐ exact source build
- ☐ allowed changes

- ☐ scope protection
- ☐ new build
- ☐ QA/Admin

H. Final Lock

- ☐ explicit client approval
- ☐ exact build
- ☐ locked status
- ☐ old builds preserved

I. Security

- ☐ project/tenant scope
- ☐ no secrets
- ☐ artifact access
- ☐ safe mock data
- ☐ audit

J. Reliability

- ☐ idempotent build request
- ☐ retry
- ☐ worker crash recovery
- ☐ artifact upload recovery
- ☐ blocker visibility

K. Admin Panel

- ☐ overview
- ☐ coverage
- ☐ build history
- ☐ artifacts
- ☐ limitations
- ☐ QA
- ☐ Admin/client review
- ☐ revisions
- ☐ blockers

L. Testing

- ☐ unit
- ☐ build
- ☐ integration
- ☐ QA loop
- ☐ security
- ☐ recovery
- ☐ E2E
- ☐ negative
- ☐ regression

25. Final Locked Prototype Agent Flow

LOCKED CLIENT-APPROVED UI → PROTOTYPE AGENT VALIDATES INPUT → BUILD PLAN → IMPLEMENT
SCREENS/ROUTES/INTERACTIONS/STATES → CREATE VERSIONED BUILD → PROTOTYPE QA → DEFECT? FIX +
RETEST → QA PASS → ADMIN REVIEW → EDIT? NEW BUILD + QA → ADMIN APPROVED → PM SHARES EXACT
BUILD → CLIENT CHANGE? PM REVISION REQUEST → PROTOTYPE AGENT NEW BUILD → QA → ADMIN → PM

Locked Prototype Agent principle: the agent creates and revises the review prototype from the exact approved UI, but it never self-approves, never fabricates production readiness, never silently expands scope, and never replaces Phase 5 full product development.